Hyper Twin Phase 3 Logic Complete

1. Architectural Overview

The Core Concept: "Plan Once, Execute Often"

To achieve high scalability (10,000+ agents), we decouple **Intelligence** (Path Planning) from **Mechanics** (Movement). Robots do not think every frame; they think only when necessary.

The Stack & Roles

- **The Backend (Tokio):** The "Brain." Handles all logic, pathfinding, and state management. It runs at a fixed tick rate (e.g., 10Hz).
- **The Network (Zenoh):** The "Nervous System." Delivers state updates from Brain to Body. Uses **Batching** to minimize syscall overhead.
- **The Frontend (Bevy):** The "Body." Purely visual. It interpolates position updates to render smooth movement for the human operator.

---

2. Key Algorithms & Concepts

A. WHCA *(Windowed Hierarchical Cooperative A*)

- **What:** A pathfinding algorithm that searches in **Space-Time** $(x, y, time)$ rather than just Space (x, y).
- **Why:** Standard A *fails in multi-agent environments because it ignores where other robots will be in the future. WHCA* guarantees collision-free paths by reserving specific tiles at specific times.
- **The "Window":** To save memory, we only plan W steps (e.g., 16 ticks) into the future.

B. The Reservation Table

- **What:** A central `HashMap<(x, y, time), RobotID>` representing the 4th dimension of the warehouse.
- **Role:** The "Traffic Controller." Before moving, a robot must query this table. If `table.is_blocked(x, y, t+1)`, the robot must wait or re-route.

C. Hybrid Flow Fields (The Optimization)

- **What:** Pre-calculated "Dijkstra Maps" for static points of interest (Chargers, Drop-off Zones).
- **Why:** Calculating $H(n)$ (Heuristic) using Manhattan Distance is a guess. Using a Flow Field value provides the *exact* travel cost around static walls.

- **Impact:** Reduces pathfinding checks from $O(b^d)$ (Exponential) to near $O(d)$ (Linear) for 90% of travel, enabling massive scale.

D. Priority & Task Bidding

- **Mechanism:** Robots obey a strict hierarchy based on `RobotID` or `Urgency`.
- **Conflict Resolution:** If Robot A (ID: 1) and Robot B (ID: 99) compete for a tile:
  - Robot A reserves it.
  - Robot B sees the reservation, fails its pathfind, and waits/re-plans.

---

3. Data Flow Cycle

1. **Task Assignment:** System assigns "Order #101" to Robot X.
2. **Planning (The Heavy Lift):**
   - Robot X locks the `ReservationTable`.
   - Runs **WHCA*** using Flow Field heuristics.
   - Reserves tiles for $t = 0$ to $t = 20$.
   - Stores path in memory: `[(x1,y1), (x2,y2)...]`.
3. **Execution (The Tick Loop - 10Hz):**
   - Robot X reads `next_step` from memory.
   - Checks immediate physical clearance (Reflex).
   - Updates `Position`.
   - Batches update into `SystemUpdate` packet.
4. **Visualization:**
   - Bevy receives batch.
   - Lerps Robot X entity to new coordinate.

---

4. Comprehensive Task Checklist

🏁 Milestone 3.1: The Single Robot Lifecycle (Vertical Slice)

_Goal: Prove the state machine works for one agent.

- [ ] **State Machine:** Implement `RobotState` enum (`Idle`, `MovingToPickup`, `Picking`, `MovingToDrop`, `LowBattery`, `Charging`, `MovingToStation`, `Blocked`, `Faulted`).
- [ ] **Task System:** Create a simple queue that assigns "Target Shelf" to Idle robots.
- [ ] **Navigation:** Implement `pathfinding` crate (Standard A*) to move Robot → Shelf → Dock.
- [ ] **Battery:** Implement drain. Trigger "Go to Charger" at 20%.

🚧 Milestone 3.2: The Reservation System (WHCA*)

Goal: Enable multiple robots to exist without ghosting through each other.

- [ ] **The Table:** Implement `ReservationTable`. Add `reserve_path(Vec<(x,y,t)>)` method.
- [ ] **Time Logic:** Implement a global `Tick` counter in Tokio.
- [ ] ***WHCA*:** Modify A* to check `is_reserved(x, y, t)` during neighbor expansion.
- [ ] **Validation:** Spawn 2 robots with crossing paths. Verify they do not collide (one should wait or go around).
- [ ] **Visualization:** Add a debug toggle to visualize robot paths and reserved tiles in Bevy (Red squares).

⚡ Milestone 3.3: The Hybrid Optimization

Goal: Performance tuning for scale.

- [ ] **Field Generator:** Write a system that bakes "Dijkstra Maps" for static POIs (Dropoff, Chargers) at startup.
- [ ] **Heuristic Injection:** Feed these maps into the A* heuristic function.
- [ ] **Benchmark:** Compare calculation time of Milestone 3.2 vs 3.3.

🛡️ Milestone 3.4: Resilience & Batching

Goal: Handling chaos and network efficiency.

- [ ] **Batching:** Refactor `zenoh_publisher` to send `Vec<RobotUpdate>` instead of single structs. (publisher sends all updates in 1 packet per tick.)
- [ ] **Obstacles:** Add mouse-click to "Block" a tile in Tokio.
- [ ] **Invalidation Logic:** When a tile is blocked, identify which reservations intersect it and force those specific robots to `Replan`.